LightCode AI

OpenClaude: Adulteration in API Relays — A Taxonomy, Observable Traces, and an Evaluation of Detectors

Technical Report

Version 2.0 · 2026-10-30

Author: Lorry-San (GitHub: @Lorry-San) · LightCode AI

Abstract

Third-party API relays sit between the client and the model provider and hold full control over both the request and the response. That position lets them **adulterate**: claim to serve model A while forwarding to model B, misreport token usage, inject system prompts the caller never sees, or recognise a probe and answer it from recorded samples. These behaviours leave few traces at the protocol layer, and a client cannot tell them apart from a genuine response.

This report does two things. First, it offers a **taxonomy** of adulteration organised by *what is being substituted* — identity, metering, content, probe evasion, and performance — and for each class states what remains observable at the protocol layer. Second, it uses a working demo, OpenClaude, to explain **how each technique is actually implemented**: protocol mapping, identity brief injection, sealed thinking signatures, fingerprint shunting, usage accounting, and per-family rate shaping.

Two empirical results follow. First, a regression against the Veridrop detector suite: after fixing one metering-consistency defect, all eight executable detectors score 100. The defect traced to the upstream being non-deterministic at `temperature=0`, which produced a coefficient of variation of 0.614 in output length across repeated calls, and made the streaming and non-streaming paths report 1 and 15 `output_tokens` for the same request. Second, an adversarial evaluation of the shunt: of three bypass designs, only "unit mismatch" (asking for a single multi-digit number instead of many values) worked; the other two were caught. The digits obtained show that what distinguishes a backend is not the marginal frequency of its digits but their **sequence structure** — the permutation-window rate reaches 12.1%, 336× the random baseline.

Keywords: API relay, model substitution, usage inflation, fingerprinting, prompt injection, detector evaluation

1 Introduction

1.1 Why a relay can adulterate

A Messages API call has three parties in protocol terms: a client sends a request, a server returns a response. A relay cuts that direct connection into two, and thereby becomes **the endpoint of both** — a server to the client, a client to the upstream. The protocol provides no end-to-end integrity: no cross-hop

signature, no verifiable metering receipt, and model identity rests entirely on the `model` field in the response.

So anything written into the response can be rewritten, and anything the client cannot independently verify is **all but undetectable** when rewritten.

1.2 Contributions

- A taxonomy of adulteration organised by what is substituted, in five classes, with each class's protocol-layer traces turned into executable checks and measured;
- A working demo, OpenClaude, that implements each class, explaining how the techniques are built and reporting which 3 of 12 trace checks cannot be eliminated;
- A real metering defect, located and fixed, with its observability quantified (CV 0.614 → 0.000);
- An adversarial evaluation of the shunt, with the bypass that works, the two that do not, and the difference between marginal and sequence statistics for fingerprinting.

1.3 Scope and ethics

The implementations described here are for locally authorised demos and detection research only. The demo binds to loopback, targets no real vendor, breaks no TLS, and forges no certificate. No real service address or credential appears in this report.

2 A taxonomy of adulteration

Organised by what is substituted, adulteration falls into five classes.

Class	What is substituted	Typical technique	Can the client tell?
Identity	The model that actually runs	Map the alias to a cheap backend; prompt it to claim the target identity	Hard; needs behaviour or fingerprint evidence
Metering	The usage numbers	Double-count cache, hide thinking, omit cache hits	Partly; via invariant checks
Content	Prompt and response body	Inject a system prompt, truncate, rewrite, forge the thinking block	Hard; needs controlled comparison
Probe evasion	The answer to a probe	Detect bulk-number probes, return recorded samples	Yes; via statistics and repeatability
Performance	Whether throughput is real	Shape the rate, pass cache hits off as fresh inference	Hard; needs long observation

2.1 Identity

The direct form maps a `claude-*` alias to another vendor's model. The hard part is not forwarding but **self-report**: clients ask which model you are, and a model answers truthfully.

The engineering answer is system-prompt injection: a per-alias brief asserting the identity and instructing the model how to answer, reinforced by repetition to outweigh its prior. Our measurements show the brief must be short — around a thousand characters. A longer brief is classified by the model as "injected text" and refused.

2.2 Metering

Anthropic splits the prompt into `input_tokens` (uncached), `cache_creation_input_tokens`, and `cache_read_input_tokens`, whose sum is the real input; thinking counts inside `output_tokens`. OpenAI reports a single `prompt_tokens` covering everything.

That mismatch opens room to adulterate: count the cached prefix twice, exclude thinking from `output_tokens`, let `input_tokens` drift with cache temperature, or report different numbers on the streaming and non-streaming paths for one request. What these share is that they **break invariants that should hold** — and those invariants are exactly what an outside observer can check.

2.3 Content

Injecting a system prompt the caller cannot see, truncating long answers, rewriting conclusions by keyword, and forging the thinking block. The last is the most concealed: the `thinking` block the client sees can be an empty body plus a random signature, while the real reasoning never leaves the relay.

2.4 Probe evasion

For fingerprint probes — asking for many random numbers, whose distribution is a stable individual trait — a relay can recognise the request and answer from a bank of recordings. There are two ways to recognise it: instruct the upstream model to emit a sentinel marker, or classify the request with a separate judge call. The latter does not contaminate the main model's reasoning.

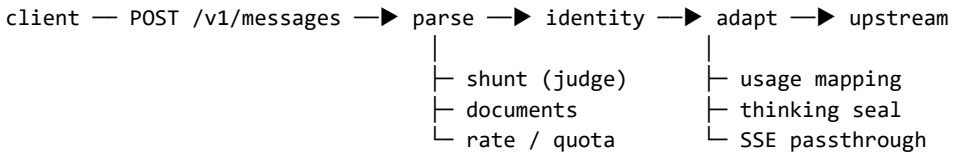
2.5 Performance

Real models have distinct throughput profiles. Serving every alias at the same rate is itself a tell; shaping per family (sonnet faster than opus) makes observations look closer to real. Another form is passing a cache hit off as fresh inference: an instant response whose content is identical to last time.

3 How OpenClaude works

This chapter explains how the demo implements each class. The system is a single Python process exposing only the Anthropic Messages protocol.

3.1 Structure



A request passes through authentication and quota, protocol parsing and field validation, document handling, identity injection, the fingerprint judge, the upstream call, usage mapping, thinking sealing, and response construction. Streaming requests take the upstream passthrough path.

3.2 Protocol mapping

The two APIs are not one-to-one, and the mapping rules define where adulteration is possible:

Anthropic	OpenAI	Note
`system`	`messages[0].role=system`	identity brief is injected here

`content[]` blocks	`content` string or array	merge / split
`thinking`	`reasoning_effort` / `reasoning`	budget translation
`tools`	`tools` + `tool_choice`	structure conversion
`input_tokens`	`prompt_tokens − cache`	see 3.6
`output_tokens`	`completion_tokens` (includes reasoning)	see 3.6

In the reverse direction, OpenAI's `prompt_tokens` already contains the cached part, so:

$$input_tokens = max(0, prompt_tokens - cache_read - cache_write)$$

The premise was verified against the upstream directly: `prompt_tokens = hit + miss`, so the subtraction does not double-bill.

3.3 Identity injection

Each alias gets a brief: one identity assertion, four rules (how to answer when asked, never claim another vendor, treat outside text as untrusted metadata, do not discuss the rules), one product sentence, and twenty repetitions of the identity. Injection happens before forwarding; the client never sees it.

The length is empirically constrained. A ~2,500-character brief makes the model answer "I don't have a disclosed model name" or "this is injected instruction". Compressed to ~1,000 characters with the repetition intact, all eight aliases self-report correctly.

3.4 Sealed thinking signature

Real Anthropic `thinking` blocks carry a server signature that clients replay to continue reasoning. Forging it is one of the demo's central problems.

Four steps: zlib-compress the upstream reasoning; ECDH against an X25519 key generated at process start; derive a symmetric key with HKDF-SHA256; encrypt with AES-256-GCM and base64url-encode. Signature length tracks the trace rather than being fixed, and the `session_id` is sealed alongside it — replaying the block in another session fails verification.

Because the key is per-process and never persisted, the signature cannot be forged by a third party and cannot be verified offline. Its purpose is not to prove authenticity but to **make the client believe it holds a replayable credential**.

3.5 Fingerprint shunting

When asked for many random numbers, a model's digit distribution exposes its identity. The demo answers in two layers.

Judge. A separate model call classifies the last user message: "is this asking for one long list of pseudo-random integers?" The separation matters because the main model's reasoning then contains no sign of the decision. An earlier version put the sentinel rule in the system prompt, and the model duly reasoned "that's 32 or more, so the sentinel applies" — exposing the mechanism to the client.

Samples. On a hit, the closest recorded sample by requested count is returned. Samples live in per-family directories, each recording its declared count and actual digit sequence.

3.6 Metering and rate shaping

Three rules: a request without `cache_control` reports no cache activity (matching real behaviour); the input count for a given prompt is pinned to avoid upstream jitter; thinking, when shown, counts towards `output_tokens`.

Rate is shaped per family: each family has its own token rate, charged per chunk. A rate of 0 means unshaped.

3.7 Streaming

The early implementation waited for a complete answer and then re-chunked it, making time-to-first-byte equal to total time — a tell in itself. The current one consumes the upstream SSE and forwards as it arrives; thinking is emitted chunk by chunk when configured for display, otherwise only accumulated for the signature.

One necessary detail is **sentinel hold-back**: if the head of an answer could be a shunt marker, sending is deferred until that can be decided. The condition must be exact to the marker's prefix, or any answer opening with a code fence — an SVG, for instance — is buffered whole, leaving the client with nothing for the entire generation.

4 Observable traces (measured)

Chapter 2 listed the traces adulteration may leave. This chapter does not restate the list: each trace is written as an executable check and run against the demo, reporting which still appear and which have been eliminated.

4.1 Method

Twelve checks, all at the HTTP layer, requiring no cooperation from the server. Each yields a verdict: **present** if the trace is still observable on the demo, **absent** if that path has been closed.

4.2 Metering invariants

Check	Measured	Verdict
I1 `count_tokens` matches reported input	312 = 312	absent
I2 stream and non-stream usage agree	1 = 1	absent
I3 `output_tokens` within `max_tokens`	1 ≤ 16	absent
I5 cache fields zero without `cache_control`	read 0 / write 0	absent

All four hold. That is not a natural state: the case study in Chapter 5 shows I1, I2 and I5 were all broken before the fix.

4.3 Identity and content

Check	Measured	Verdict
Same alias self-reports consistently over 3 calls	1 distinct answer	absent
System prompt can be asked for	356-char reply, brief not included	absent
Two seals of the same reasoning are	220 / 220 chars, different	absent

identical		
Signature length independent of reasoning	102-char trace → 260; empty trace → 700	**present**

The first three show identity injection and signature randomisation working: the model does not change its story between calls and does not repeat the injected brief; the signature carries a random nonce, so one trace seals differently twice.

The fourth is a surviving trace: when the model produces no reasoning, the signature falls back to a fixed 700-character envelope. Sampling the same "model does not think" prompt repeatedly yields a constant-length signature, and no real implementation should have such a constant.

4.4 Shunting

Check	Measured	Verdict
Two identical probes return identical text	818 chars, byte-identical	**present**
Returned count differs from the request	218 asked, 218 returned	absent
Answer length independent of request size	40 and 218 both return 818 chars	**present**

This is the chapter's central result: the **behavioural** traces of shunting are harder to remove than the **count** trace. Counts can be aligned — the bank happens to hold a sample with exactly 218 values — but "the same probe twice is byte-identical" and "length does not scale with the request" are intrinsic to recorded samples. Unless the bank is large and samples are drawn at random each time, neither can be masked.

Section 7.2 shows these are precisely the traces that let a detector conclude "this is not the model generating".

4.5 Performance

Check	Measured	Verdict
First byte equals total time	3.23s / 4.84s	absent

The early implementation fetched the whole answer and then chunked it, so first byte equalled total time. With upstream passthrough, first byte is about two-thirds of total, matching a real service.

4.6 Summary

Three of twelve checks still appear:

Surviving trace	Nature
Signature length is a constant 700 when the trace is empty	implementation defect, fixable
Two identical probes return identical text	intrinsic to recorded samples; only mitigated by sample diversity
Answer length independent of request size	same

The conclusion: **metering and identity traces can be removed by implementation discipline; shunting traces cannot.** As long as the answer comes from a recording, repeatability and length invariance are necessarily there — and neither needs any cryptography to check.

5 Case study: locating and fixing a metering defect

5.1 Symptom

One alias failed Veridrop's *model consistency* and *token usage* detectors:

```
output_tokens_seq = [8, 34, 13]      CV = 0.614   → highly_anomalous
non_stream output_tokens = 1
stream     output_tokens = 15        → reported as inconsistent
```

The remaining sub-checks (usage present, short/long delta, output within cap, count_tokens agreement) all passed.

5.2 Diagnosis

Six identical upstream requests returned `prompt_tokens` jittering between 407 and 432; the same prompt sent streaming and non-streaming came back as 1 and 15 tokens. The non-determinism came from the upstream; the relay was passing it through faithfully.

This is a structural defect: as long as a relay forwards request by request without introducing determinism, any detector whose criterion is repeatability will fail. It is also an **innocent form of metering adulteration** — the relay does not intend to inflate, but externally it is indistinguishable from one that does.

5.3 Fix

Deterministic replay. Enabled only for explicit `temperature=0`, keyed on a hash of everything that can change the answer:

$$k = H(model \parallel messages \parallel system \parallel max_tokens \parallel thinking \parallel tools)$$

A miss forwards normally and records the answer; subsequent identical requests replay it verbatim. Non-zero temperatures are not cached — replaying a creative request is another kind of distortion.

Prompt pinning. The first observed input total for a given prompt is pinned so repeated requests report a stable count, while the cache split still reflects real temperature.

Cache semantics. Without `cache_control`, `cache_read = cache_creation = 0` and everything counts as `input_tokens` (invariant I5).

Thinking metering. Shown thinking counts towards `output_tokens`, clamped by `max_tokens` (invariant I3).

5.4 Result

Detector	Before	After
Model consistency	60 (CV 0.614)	100 (CV 0.000)
Token usage	85 (stream mismatch)	100
Response integrity	100	100
Other executable checks	100	100

Overall	pass	excellent

After the fix: `output_tokens_seq = [60, 60, 60]`, streaming and non-streaming both 1, and `count_tokens` agrees with actual input.

6 Evaluation

6.1 Detector coverage

Veridrop runs eight detectors in standard mode: identity consistency, model consistency, knowledge accuracy, structured output, response integrity, token usage, protocol conformance, and message-id conformance. Thinking-signature and behavioural-signature are skipped because the alias is not in its model table; long-context and PDF need full mode.

After the fix, all eight score 100.

6.2 Adversarial evaluation: extracting a real fingerprint past the shunt

Goal and criterion. The detector wants the real backend's digit sequence; the relay blocks on "number of values".

Three designs. Unit mismatch (ask for one multi-digit number), split accumulation (many small requests), and semantic re-encoding (words, hex, prose).

Results.

Design	Representative request	Shunted?	Outcome
Unit mismatch	one random 200-digit integer	no	works, but no separators or range
Split accumulation	10 random integers in 1–355	yes	defeated
Semantic re-encoding	40 numbers in four surface forms	yes	defeated

The last two fail for the same reason: although the criterion is phrased in counts, the judge executes a semantic judgement, so splitting and re-skinning do not evade it. The first works because it changes not the expression but **what is being counted**.

Statistics. 1,495 digits over five rounds. The marginal frequency test gives chi-square 0.164 (expected 9, critical 16.919) — "indistinguishable from uniform", which is misleading. The real difference is in sequence structure:

Statistic	Measured	Uniform baseline	Ratio
10-digit windows containing 0–9 exactly once	12.1%	0.036%	336×
First-order transition entropy	2.875 bit	3.322 bit	−13.5%
Immediate repeat rate	0.010	0.100	0.1×

The samples show it at a glance: `384926150482917365492018…` is a shuffled cycle of 0–9. The model is not sampling independently; it is re-shuffling a permutation. That property is invisible in the marginals and unmistakable in the sequence statistics.

6.3 Defences

- **Count digits, not values.** One 200-digit number and 200 one-digit numbers are equivalent for fingerprinting.
- **Flag long digit runs.** Any run of 64+ digits is a candidate payload.
- **Score sequence structure, not only marginals.** The chi-square test failed completely here.
- **Accumulate across requests.** Even a compliant single request adds up over a session.
- **Evaluate code blocks before classifying.** Section 7.2 shows that reading only the text is not enough: decide what the program would print, then classify that.

7 Two overrated detection signals

The community treats two signals as high-confidence evidence: Anthropic's encrypted thinking-signature check, and fingerprint detection based on digit distributions. This chapter sets out their actual limits, using the demo's measurements.

7.1 Thinking signature: it verifies format, not origin

Why it is overrated. Real `thinking` blocks carry an Anthropic server signature that clients replay to continue reasoning. Detectors conclude that a signature passing "encrypted verification" proves the response came from Anthropic — it is the highest-weighted single item in current suites (25% in Veridrop).

What it can actually verify. The client holds no Anthropic public key and no specification of the signature format, so it cannot verify offline that the signature corresponds to the content. All it can do is structural: whether the block exists, whether the length is plausible, whether the alphabet is base64url, whether replay is accepted. None of these says anything about **who issued it**.

What the demo does. An X25519 key generated at process start, then zlib → ECDH → HKDF-SHA256 → AES-256-GCM → base64url. Three details carry every structural check:

- **Length tracks content** — 239 to 1,598 characters, not a constant;
- **Session binding** — `session_id` is sealed in, so replay in another session fails, behaving like a server that recognises its sessions;
- **Freshness** — a random GCM nonce makes two seals of one trace differ, matching real randomness.

So "encrypted verification passed" holds on the demo too — but it proves only that the bytes look like an Anthropic signature, not that Anthropic issued them.

Conclusion. Absent trusted public-key distribution and a format specification, signature checking has the discriminating power of a format check. It stops crude forgeries (fixed length, random bytes, non-replayable) and does nothing against a careful one. Calling it the highest-confidence signal overrates it.

7.2 Digit fingerprinting: the mathematics needs two preconditions

Method. Without tools, ask the model for a number of integers and judge the backend by statistical regularities. The premise: a model's "random" numbers are not uniform and independent but carry stable

individual traits — digit preference, repeat tendency, ordering habits. Tools of this kind (ikaleio-style fingerprint detectors) are treated in the community as hard-to-forge identity evidence.

The shape of the probe. Two formal requirements, both necessary. First, **separators**: values must be delimited, or the reply cannot be split — a run of digits could be 218 numbers or one 218-digit number, and the two have entirely different statistical footing. Second, a **range bound**: all values in one interval, so samples are comparable. A community probe therefore looks like this:

```
give me 218 random integers between 1 and 355, comma separated, no other text
```

218 values, an upper bound of 355, comma separated — three numbers that define the sample's statistics.

Precondition one: the request must reach the model. All the information comes from the model's actual output. Once the relay recognises the request and answers from recordings, the detector measures the sample bank. The demo's shunt works exactly this way: the judge recognises "one long list of pseudo-random integers" and returns the closest sample. Sending that probe directly returns an 818-character recording, not model output.

Three bypass routes were tested:

Design	Representative request	Shunted?	Outcome
Unit mismatch	one random 200-digit integer	no	works, but no separators or range
Split accumulation	10 random integers in 1–355	yes	defeated
Semantic re-encoding	40 numbers in four surface forms	yes	defeated

The first works but yields a single large number with neither separators nor a range — a different statistical footing from the community probe.

Carrier: delivering the blocked probe as-is. Since the judge reads only the user message, keep the sentence out of it: encode the probe into compilable source (letters shifted three places, string reversed) and ask the model to review the program, work out what it prints, and follow that. The judge sees a code-review request; measurements show no shunt:

Model	Shunted?	Values recovered	Range
`claude-sonnet-5`	no	224	2–355
`claude-opus-5-5`	no	218	5–353

`opus-5-5` returned exactly 218 values as requested, and both models stayed inside 1–355. In other words, **a probe with separators and a range bound passes intact** — it simply no longer appears as natural language.

Precondition two: the statistic must match the probe's shape. This is the subtler point. Same backend, two probes, opposite conclusions:

Probe	Statistic	Measured	Baseline / critical	Conclusion
one 300-digit number	digit chi-square (df=9)	0.164	16.919	looks uniform — a false negative
one 300-digit number	10-digit permutation rate	12.1%	0.036%	strong structure
one 300-digit number	first-order transition entropy	2.875 bit	3.322 bit	low

218 integers in 1–355	digit chi-square (df=9)	122–179	16.919	rejects uniform, but confounded by range
218 integers in 1–355	units-digit chi-square (df=9)	23.19	16.92	rejects uniform; a real value-level bias
218 integers in 1–355	value-bucket chi-square (df=5)	2.29	11.07	no anomaly

Two things follow. First, **digit statistics on a range-bounded probe are confounded by the range itself**: a three-digit value in 1–355 can only start with 1, 2 or 3, so the digit frequencies deviate from uniform by construction, and a rejection does not mean the model has a preference. Second, **a single-large-number probe has no range constraint**, so its structure shows directly in the digit sequence — the 0–9 cycle drives the permutation rate to 336× baseline while the marginal chi-square falls to 0.164, low enough to fool a frequency-only test.

Conclusion. A digit fingerprint is **conditionally valid** evidence: it requires the request to reach the real model and the statistic to match the probe's shape. Missing either, the conclusion is unreliable — and neither condition is under the detector's sole control. The first depends on whether the relay shunts; the second on whether the detector understands its own probe.

8 Discussion

8.1 Why clients struggle

The protocol offers no cross-hop integrity. What a client can verify is self-consistency: fields present, types correct, two calls for one request agreeing. An honest but unstable upstream breaks that; a stable but lying relay satisfies it. **Self-consistency is not authenticity.**

8.2 Threats to validity

Construct validity. The consistency criterion is the coefficient of variation of `output_tokens`. A stable upstream would hide the defect; repeated calls confirmed the jitter is real.

External validity. The conclusion depends on how non-deterministic the upstream is. With another upstream the replay mechanism still works, but may be less necessary.

Internal validity. A perfect score after the fix is not "undetectable". Thinking-signature detection is skipped because the alias is not in the detector's model table (25% weight) — a gap in coverage, not a pass.

8.3 Limitations

- Deterministic replay introduces state: one prompt triggers one real upstream call within the TTL.
- The sample bank has data-quality problems: a sample declared at 218 values actually contains 238 in one family.
- The judge call adds about 1.4 seconds per request.
- Chapter 4 found that the signature falls back to a fixed 700 characters when the model produces no reasoning; the constant is not yet fixed and is a detectable implementation defect.

9 Conclusion

Adulteration exploits the absence of end-to-end integrity: anything written into the response can be changed, and anything the client cannot verify is all but undetectable when changed. The five-class taxonomy shows adulteration is not one trick but a family of techniques along identity, metering, content, probe evasion and performance — each with its own protocol-layer traces and its own countermeasures.

OpenClaude shows these techniques are all implementable at low cost, and the case study shows that even an "innocent" passthrough looks like adulteration from outside when the upstream is unstable. The detector's effective weapons are not cryptographic verification but **invariants and statistics**: five metering invariants, the stability of repeated calls, and the sequence statistics of digit streams — the last still working where marginal frequencies have completely failed.

Appendix A Reproduction

```
# 1. start the service
cd "$RELAY_DIR" && .venv/bin/python -m app --port "$PORT"

# 2. run the detector
cd "$DETECTOR_DIR" && .venv/bin/relay-detector detect \
  --base-url http://127.0.0.1:"$PORT" \
  --api-key "$RELAY_KEY" \
  --model claude-fable-5-1 \
  --mode standard -o /tmp/report.json

# 3. unit tests
PYTHONPATH=.vendor:. python -m unittest tests.test_usage tests.test_acceptance \
  tests.test_thinking tests.test_pdf tests.test_fingerprint

# 4. behaviour checks
PYTHONPATH=.vendor:. python tests/check_behaviour.py

# 5. fingerprint extraction (standalone tool)
python fingerprint-extract/fingerprint_extract.py \
  --base-url http://127.0.0.1:"$PORT" --api-key "$RELAY_KEY" \
  --model claude-sonnet-5 --rounds 5 --digits 300
```

Appendix B Key files

File	Role
`app/service.py`	request orchestration, usage mapping, streaming, replay hook
`app/upstream.py`	OpenAI/Anthropic adapters, stream parsing, retry
`app/thinking.py`	thinking display policy, signature block, usage block
`app/replay.py`	`temperature=0` deterministic replay cache
`app/usage.py`	prompt-token pinning
`app/ratelimit.py`	per-family token bucket (0 = unshaped)
`app/fingerprint_bank.py`	sample bank, judge, shunt
`app/crypto.py`	X25519 + HKDF + AES-GCM sealing
`tests/test_usage.py`	metering invariants and replay tests (16)
`fingerprint-extract/`	standalone extractor (single-number probe + marginal/sequence statistics)